

Sistemas Operativos

1º Parcial 1C2026 TT – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

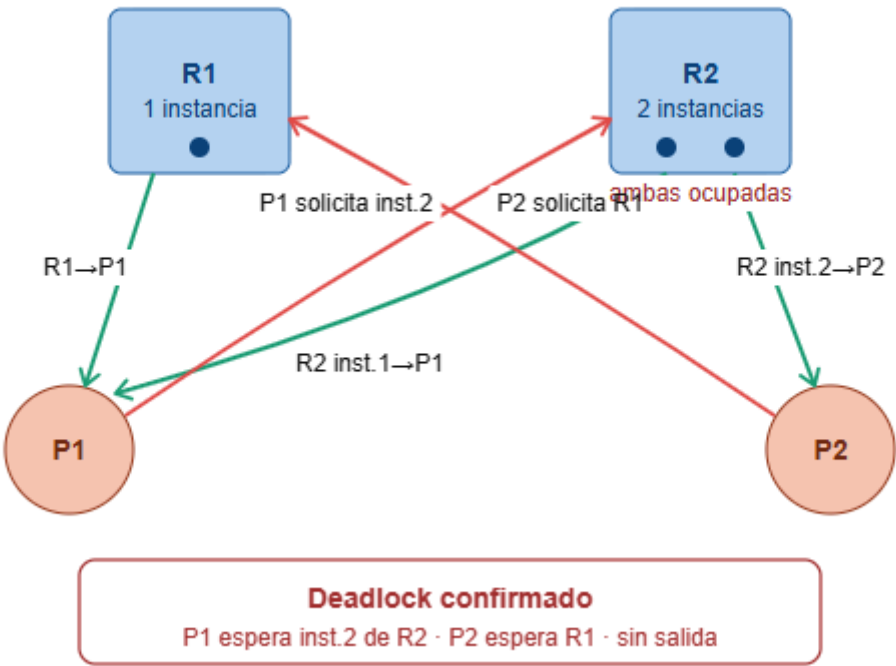
- 1-
 - a) Estas son las interrupciones **Enmascarables**.
 - b) Desde el inicio hasta que se comienza a atender la 2da interrupción ocurren **2 cambios de contexto y un cambio de modo**.
 - c) (o d en 2do tema) Además del código y los datos estáticos, los elementos que conforman la imagen del proceso son: **Heap** (Memoria dinámica), el **Stack** (Pila del sistema), y el **PCB** (Estructura del SO, que administra el proceso).
 - d) (o c en 2do tema) **Livelock**, ya que los procesos siguen ejecutándose aunque no pueden progresar.
- 2-
 - i) (d-b) Variables ubicadas en la pila (stack)
 - ii) (b o c en 2do tema) Nunca se puede producir starvation
 - iii) (d o a en 2do tema) Syscalls potencialmente bloqueantes
 - iv) (c o b en 2do tema) Detectar deadlocks actuales
- 3- El SO usa el PCB para almacenar el estado y contexto de la ejecución del proceso, así como también los punteros a memoria correspondientes al código, datos, y Heap.
Si el sistema tiene hilos KLTs, se tiene un PCB, con los punteros a memoria correspondiente a los datos, código y heap, y además va a tener un TCB por cada hilo con la información correspondiente a su contexto de ejecución, su stack y su estado individual.
- 4- Es el nivel de multiprogramación (cantidad de procesos alojados en memoria para ejecutarse en un momento determinado).
Los planificadores influyen para aumentar o disminuir su carga, depende de cómo se administran los procesos. El de largo plazo es el que más influye, controlando cuantos procesos ingresan al sistema (controla el nivel de multiprogramación). El de mediano plazo regula la carga suspendiendo o activando procesos.
El estado Exit tiene la funcionalidad de guardar, temporalmente, información sobre un Proceso que ha finalizado su ejecución. Dos ejemplos de esto son: guardar estadísticas de ejecución del Proceso (tiempo de ejecución, etc), o guardar el estado de retorno del mismo (success, error, etc)

5-La siguiente tabla es una comparacion entre ambas estrategias.

	Prevención	Evasión
Estrategia	Se asegura que en el sistema nunca suceda un deadlock, eliminando una de las condiciones.	Garantiza la no-existencia del deadlock, manteniendo al sistema en un estado seguro.
Flexibilidad de peticiones de recursos	Restringida , por las políticas aplicadas para eliminar las condiciones y evitar el deadlock.	Media, ya que los procesos deben declarar sus peticiones máximas.
Overhead	Poco, ya que no necesita analizar continuamente los estados.	Alto, ya que cada solicitud requiere verificar si el estado resultante es seguro.

Grafo:

2 procesos , 2 recursos, 1 recurso con 2 instancias.

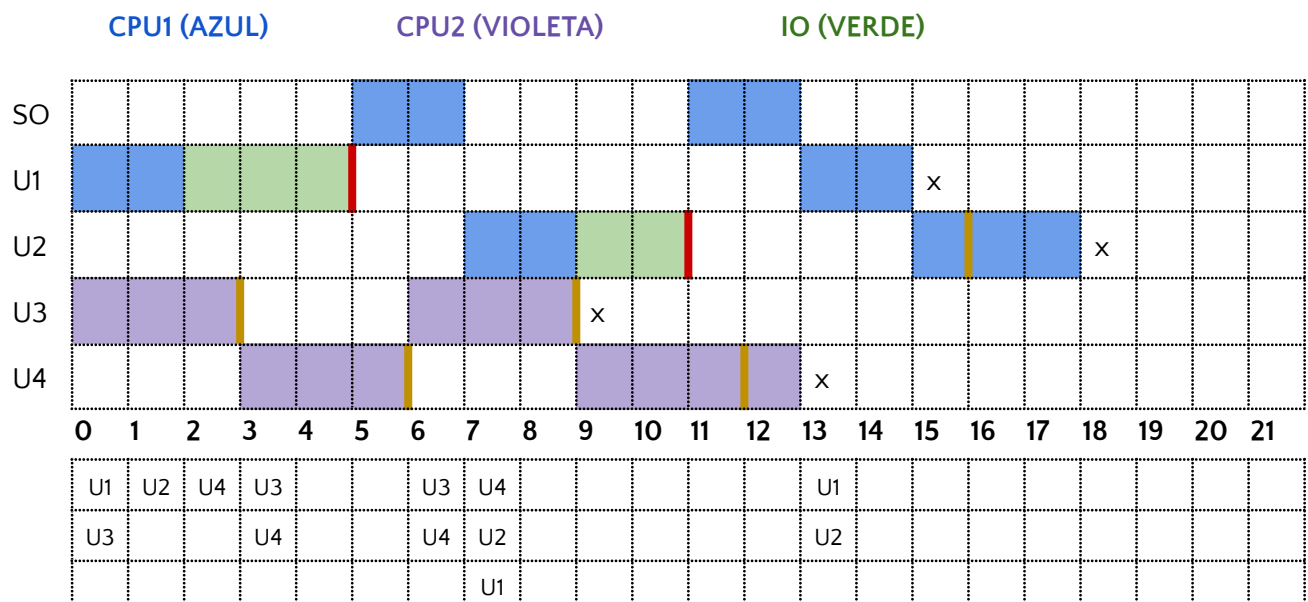


Práctica

1. **SB** = 1 (Es un mutex) **SC** = 5 **SA** = 0 (Semáforos Contadores/Binarios)

Proceso A (N instancias)	Proceso B (N instancias)
elemento = producir() wait(SC) wait(SB) depositar(elemento, lista) signal(SB) signal(SA)	wait(SA) wait(SB) elemento = retirar(lista) signal(SB) signal(SC) consumir(elemento)

2.
 - a) RR $Q=3$
 - b) KA usa SJF (sin desalojo) con Jacketing. KB usa FIFO sin Jacketing.
3. Ejercicio a desarrollar:
 - a) Los instantes marcados en línea roja son aquellos donde se produjeron interrupciones: 5, 11
 - b) Se solicitan Syscalls en los instantes: 0, 2, 9, 13, 18.
Por otro lado, la biblioteca de ULT's interviene por dos motivos: syscalls wrapeadas y planificación por simil RR en los instantes: 2, 3, 6, 9, 12, 16



4.

Primera solución:

(con 4 semáforos)

Inicialización de los semáforos:

sem_m = 3 **sem_a = 0** **sem_l = 2** **sem_g = 1**

Proceso M	Proceso A	Proceso L	Proceso G
<pre>while(1){ wait(sem_m) wait(sem_m) wait(sem_m) print("M") signal(sem_a) }</pre>	<pre>while(1){ wait(sem_a) print("a") signal(sem_l) signal(sem_g) signal(sem_m) }</pre>	<pre>while(1){ wait(sem_l) wait(sem_l) wait(sem_l) print("l") signal(sem_a) }</pre>	<pre>while(1){ wait(sem_g) wait(sem_g) wait(sem_g) print("g") signal(sem_a) }</pre>

Demostración:

Orden de sincro →	A	M	A	L	A	G	A
sem_m	3	0	1	1	2	2	3
sem_a	0	1	0	1	0	1	0
sem_l	2	2	3	0	1	1	2
sem_g	1	1	2	2	3	0	1

Segunda solución

(con 5 semáforos)

Inicialización de los semáforos:

sem_m = 1 **sem_a = 0** **sem_l = 0** **sem_g = 0** **sem_mlg = 1**

Proceso M	Proceso A	Proceso L	Proceso G
<pre>while(1){ wait(sem_m) wait(sem_mlg) print("M") signal(sem_l) signal(sem_a) }</pre>	<pre>while(1){ wait(sem_a) print("A") signal(sem_mlg) }</pre>	<pre>while(1){ wait(sem_l) wait(sem_mlg) print("L") signal(sem_g) signal(sem_a) }</pre>	<pre>while(1){ wait(sem_g) wait(sem_mlg) print("G") signal(sem_m) signal(sem_a) }</pre>

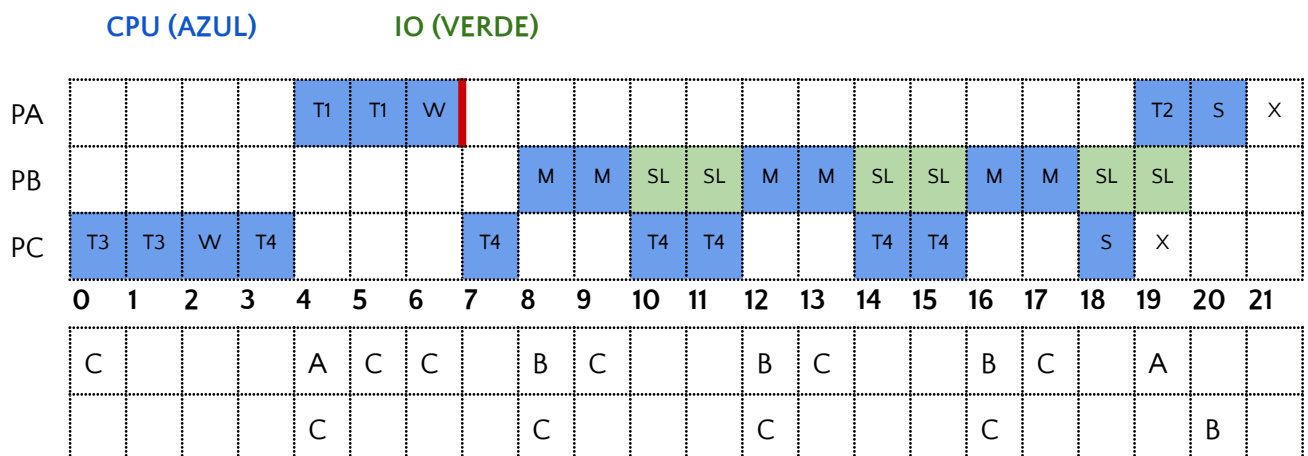
Demostración:

Orden de sincro →	A	M	A	L	A	G	A
sem_m	1	0	0	0	0	1	1
sem_a	0	1	0	1	0	1	0
sem_mlg	1	0	1	0	1	0	1
sem_l	0	1	1	0	0	0	0
sem_g	0	0	0	1	1	0	0

cantidad de mutex: 0

5.

a)



En T=7 el Proceso A quedó bloqueado a causa del semáforo.

b) Se observa que PA, que tiene la máxima prioridad, se encuentra bloqueado esperando que PC (menor prioridad) libere el recurso, y dicha situación se retrasa todavía más debido a que PB (de menor prioridad que PA) toma el procesador. Dicha situación constituye una **Inversión de Prioridades**.

Para solucionarlo, se puede aplicar Herencia de Prioridades. En este caso, PC subiría temporalmente a prioridad 0 en el instante 7 (cuando PA intenta tomar el recurso), y volvería a su prioridad original luego de ejecutar el Signal que lo liberaría.